



1. Агенту нельзя доверять больше прав, чем требует его задача

Когда мы переходим от учебного чат-бота к агенту, который умеет читать файлы, запускать команды, работать с GitHub, обращаться к API и взаимодействовать с инфраструктурой, вопрос безопасности становится частью архитектуры системы.

Я исхожу из простого правила: если агенту для выполнения задачи не требуется определённое действие, технической возможности выполнить это действие у него тоже желательно не иметь.

Например, агент-ревьюер должен иметь возможность читать исходный код, получать `git diff`, запускать тесты и линтер. Но из этого совершенно не следует, что ему нужны права на `git push`, изменение production-конфигурации или удаление файлов.

Такой подход называется принципом наименьших привилегий, или least privilege. Агент получает минимальный набор полномочий, достаточный для выполнения своей роли.

2. Доступ к коду тоже необходимо разделять

Мы часто говорим «агент имеет доступ к проекту» так, будто это одно разрешение. На практике внутри работы с кодом существует несколько принципиально разных возможностей.

Одно дело — прочитать файл. Совсем другое — изменить его. Ещё более серьёзное действие — выполнить содержащийся в проекте код или команду терминала.

Поэтому я стараюсь мысленно разделять доступ на чтение, изменение и исполнение.

Hermes использует Toolsets как основной механизм определения доступных агенту возможностей. Например, файловые операции и терминал относятся к отдельным группам инструментов, а набор доступных Tools может изменяться для конкретной платформы или сессии. Это позволяет не выдавать каждому агенту автоматически полный технический набор возможностей.

Именно поэтому read-only Reviewer и Migrator с правом изменения кода должны рассматриваться как разные роли, даже если работают с одним репозиторием.

3. Секреты не являются обычными файлами проекта

Особое отношение необходимо к секретам. API key, токен Git-хостинга, ключ SSH или пароль базы данных нельзя воспринимать как ещё один текстовый файл, который модель может прочитать «для удобства».

Чем больше компонентов получает секрет, тем больше становится поверхность возможной утечки.

В Hermes есть отдельные защитные механизмы для credential-файлов и переменных окружения. Например, защищённые пути с SSH-, AWS-, Kubernetes- и некоторыми Hermes-секретами блокируются для обычной записи, а при выполнении кода переменные окружения с характерными именами вроде `TOKEN`, `SECRET`, `PASSWORD` и `CREDENTIAL` по умолчанию отфильтровываются.

Поэтому безопасная архитектура строится не на фразе в промпте «не показывай токены», а на том, что ненужный секрет вообще не попадает в среду выполнения агента.

4. Доступ к инфраструктуре требует более строгой границы

Работа с локальным исходным кодом и работа с инфраструктурой имеют разный масштаб последствий.

Ошибка в локальной ветке обычно можно отменить через Git. Ошибка агента, имеющего доступ к production-серверу, облачной панели или базе данных, может повлиять уже на реальную систему.

Поэтому доступ к инфраструктуре я рассматриваю как отдельную доверительную границу.

Если агенту достаточно проверить состояние сервиса, лучше предоставить ему read-only операцию состояния, чем полноценный SSH-доступ к серверу. Если требуется работать с определённым API, лучше предоставить узкий Tool с конкретными функциями, чем передавать агенту универсальный административный токен.

Hermes поддерживает разные terminal backends, включая локальную среду, Docker и SSH. Изоляция контейнера может использоваться как дополнительная граница, причём в Docker-среде host environment по умолчанию не передаётся целиком.

5. Least privilege начинается с роли агента

Чтобы применять least privilege системно, сначала нужно определить роль.

Например, Reviewer должен проверять. Migrator должен изменять ограниченную часть проекта. Release Agent может взаимодействовать с CI/CD. Documentation Agent работает с документацией.

Если я сначала выдаю агенту все доступные возможности, а потом пытаюсь словами объяснить, какие из них нельзя использовать, архитектура получается слабой.

Правильнее начинать с минимального профиля возможностей и расширять его только тогда, когда появляется конкретная необходимость.

Hermes Profiles позволяют содержать независимые конфигурации для разных агентов. У каждого профиля могут быть собственные настройки, credentials, память, Skills и состояние. Это удобно для разделения специализированных исполнителей, потому что один агент не обязан автоматически наследовать среду другого.

Именно на этом принципе будет строиться следующая практическая работа с ролевой моделью.

6. RBAC в агентной системе

Теперь можно перейти к понятию RBAC — Role-Based Access Control, то есть управлению доступом на основании роли.

В классической информационной системе пользователь получает роль, а роль определяет разрешённые операции. В агентной архитектуре идея остаётся той же.

Мы можем иметь роль Reviewer, которой разрешено чтение и диагностика, и роль Migrator, которой дополнительно разрешено ограниченное изменение файлов. Отдельный оператор может иметь возможность создавать задачи и подтверждать опасные действия.

В Hermes такая модель собирается из нескольких механизмов. Профиль отделяет одного агента от другого, Toolsets определяют технические возможности, MCP-фильтры позволяют сокращать набор функций внешнего сервера, а plugin capabilities требуют отдельного согласия на привилегированные действия.

Поэтому RBAC здесь нужно понимать не как одно поле `role=reviewer`, а как согласованный набор реальных технических ограничений.

7. Опасные операции требуют отдельного контроля

Даже агент с необходимым доступом не должен автоматически выполнять любое потенциально опасное действие.

Hermes использует механизм approval для команд, которые распознаются как рискованные. В стандартных режимах опасная команда может потребовать подтверждения пользователя перед фактическим выполнением.

Разрешение может действовать только для одного вызова, для текущей сессии или сохраняться как постоянное правило. При отсутствии ответа approval по истечении времени закрывается с запретом выполнения. Такая модель соответствует принципу fail closed: если разрешение не было явно получено, операция не выполняется.

Для меня здесь особенно важно различие между полномочием и подтверждением. Агент может технически иметь terminal Tool, но конкретная опасная команда всё равно может потребовать отдельного человеческого решения.

8. Изоляция ограничивает последствия ошибки

Даже хорошо настроенный агент может ошибиться. Поэтому следующая задача безопасности заключается не только в предотвращении ошибки, но и в ограничении её последствий.

Если агент выполняет код прямо на рабочей машине пользователя с полным доступом к файловой системе, потенциальная область воздействия очень велика.

Если та же задача выполняется внутри ограниченного контейнера с минимальным набором файлов и без host-секретов, ошибка остаётся внутри гораздо меньшей области.

Hermes строит защиту по принципу *defense in depth*: одновременно используются *approvals*, ограничения файловой записи, контейнерная изоляция, фильтрация *credentials* для MCP, проверка *project context* и другие независимые слои.

Я хочу, чтобы мы воспринимали *sandbox* не как замену правильным разрешениям, а как следующий защитный слой на случай, если предыдущий контроль оказался недостаточным.

9. Аудит отвечает на вопрос «что действительно сделал агент»

Безопасность агентной системы заканчивается не в момент выполнения действия. После этого должна оставаться возможность восстановить происходившее.

Мне важно знать, какой агент запустил действие, какую команду он пытался выполнить, какие разрешения были предоставлены, какой процесс реально завершился и каким был результат.

Это и есть аудит.

В Hermes состояние разных операций сохраняется на нескольких уровнях. Сессии и решения *approval* имеют постоянное состояние, а результаты фоновых процессов сохраняются как отдельные *redacted receipts*, связанные с породившей их сессией. Для Kanban мы уже видели историю *runs* и статусы задач.

Аудит нужен не только после атаки. Он помогает разбирать обычную ошибку агента и понимать, было ли неверным решение модели, конфигурация разрешений или сам инструмент.

10. Инцидент показывает качество модели доступа

Представим ситуацию: *Reviewer*, которому требовалось только проверить изменения, каким-то образом попытался изменить *production*-конфигурацию.

Сам факт такого намерения модели ещё не должен превращаться в реальный инцидент.

Если *least-privilege* настроен правильно, у *Reviewer* вообще отсутствует нужный *Tool* или *credential*. Если операция проходит через терминал и относится к опасным, её может остановить *approval*. Если код выполняется в *sandbox*, область воздействия дополнительно ограничена. После попытки у нас остаются данные, позволяющие установить, что произошло.

Именно так нужно оценивать безопасность агента: не по тому, способен ли LLM никогда не ошибаться, а по тому, насколько система сохраняет контроль, когда модель всё-таки принимает неправильное решение.

Поэтому в следующей практической работе мы будем проверять уже не промпты. Мы создадим несколько ролей с различными полномочиями, настроим для них ограниченный доступ в Hermes, затем специально смоделируем запрещённое действие и разберём, какой слой защиты остановил его и какие следы этого события остались для аудита.
